

```
> bank <- read.csv(file="bank.csv", sep=";")

> bank[1:3,]
  age      job marital education default balance housing
loan contact day
1 30 unemployed married primary      no    1787      no
no cellular 19
2 33 services married secondary    no    4789      yes
yes cellular 11
3 35 management single tertiary    no    1350      yes
no cellular 16

  month duration campaign pdays previous poutcome y
1 oct      79      1    -1      0 unknown no
2 may     220      1   339      4 failure no
3 apr     185      1   330      1 failure no
```

The 'data' and 'x' or 'y' arguments are required to produce a bar chart. If either one is not provided, an error is thrown, as shown below:

```
> library(ggplot2)

> qplot(data=bank)
Error: stat_bin() requires an x or y aesthetic.
Run `rlang::last_error()` to see where the error occurred.
```

You can view a backtrace of the previous error in an R session using 'rlang::last_error()' invocation, as illustrated below:

```
> rlang::last_error()
<error/rlang_error>
stat_bin() requires an x or y aesthetic.
Backtrace:
 1. (function (x, ...) ...) ...
 2. ggplot2::print.ggplot(x)
 3. ggplot2::ggplot_build.ggplot(x)
 4. ggplot2::by_layer(function(l, d) l$compute_statistic(d,
layout))
 5. ggplot2::f(l = layers[[i]], d = data[[i]])
 6. l$compute_statistic(d, layout)
 7. ggplot2::f(..., self = self)
 8. self$stat$setup_params(data, self$stat_params)
 9. ggplot2::f(...)
Run `rlang::last_trace()` to see the full context.
```

The minimum argument required for plotting the data is as follows:

```
> qplot(x=job, data=bank)
```

Cannot open connection

If we run the R session in a different directory where the 'bank.csv' file does not exist, we will get a 'No such file or directory' error when trying to open a connection to read the file.

```
> bank <- read.csv(file="bank.csv", sep=";")
Error in file(file, "rt") : cannot open the connection
In addition: Warning message:
In file(file, "rt") :
  cannot open file 'bank.csv': No such file or directory
```

Condition handling

You can program defensively, and R provides the following functions to try and catch errors in your source code.

stop(): The *stop()* function terminates the current execution of the code. For example:

```
> a <- FALSE
> if (a == FALSE) { stop("False!") }
Error: False!
```

try(): The *try()* function in R can attempt to validate an expression and then executes the following code block:

```
> try(if(a == FALSE) { stop("False!") })
Error in try(if (a == FALSE) { : False!
```

tryCatch(): You can use the *tryCatch()* function to include specific warning and error handling routines for your code. An example illustrating warning and errors is given below:

```
> tryCatch( {Shingle()} , warning = function(w) {
print("Warning!")}, error = function(e) { print("Error!" )})
[1] "Error!"
> a <- "Hello World"
> tryCatch( {as.numeric(a)}, warning = function(w) {
print("Warning!")}, error = function(e) { print("Error!" )})
[1] "Warning!" END 
```

References

- [Moro et al., 2014] S. Moro, P. Cortez and P. Rita, A Data-Driven Approach to Predict the Success of Bank Telemarketing. Decision Support Systems, Elsevier, 62:22-31, June 2014
- Error handling in R, <https://www.r-bloggers.com/2012/10/error-handling-in-r/>

 **By: Shakthi Kannan**

The author is a free software enthusiast.